IceFEM: A finite element code to simulate fracture and viscous/plastic deformation in ice.

Tim Hageman*

Department of Engineering Science, University of Oxford, Oxford OX1 3PJ, UK

Abstract

Documentation that accompanies the *C++* code IceFEM, available from here. This documentation explains how to compile the code, the usage of the implemented finite element framework, and highlight the main files. The code allows the simulation of the detailed failure behaviour of ice, considering brittle fractures through the phase-field model, capturing shear bands through von Mises plasticity, and capturing creep through Glens law. Several example files are also included, demonstrating how to set up the simulation of a tri-axial compression sample, and of an ice-cliff failure study. If using (part of) this code, please cite T Hageman, C Andrade, and E Martinez-Pañeda. *Corrosion of metal reinforcements within concrete and localisation of supporting reactions under natural conditions.* Acta Electrochemica [1].

Keywords: Glaciology, Finite elements, Phase-field fracture, Plasticity, Ice mechanics

*Corresponding author

Email address: `tim.hageman@eng.ox.ac.uk` (Tim Hageman)

Contents

1 Introduction **1**
1.1 Installation . . . 1
1.1.1 Required external libraries . . . 1
1.1.2 Compiling . . . 1
1.2 Basic usage . . . 1
1.3 Input Files . . . 1
1.3.1 General settings . . . 2
1.3.2 Mesh settings . . . 2
1.3.3 Material settings . . . 2
1.3.4 Solver settings . . . 4

2 References **5**

1. Introduction

1.1. Installation

1.1.1. Required external libraries

The simulation code has been developed to work directly in ubuntu, or within windows using WSL [2]. A script is included which installs all required libraries, *Install_ICEFEM.sh*. While it can be directly run to install and compile all libraries at once, it is strongly recommended that the contents is run line-by-line by copying it into the terminal. This allows for easier debugging in case of errors. The following libraries are required:
Intel OneMKL [3] is used for its linear matrix solvers (and used by PETSc).
Eigen [4] is used for small matrix and vector mutliplications.
RapidJSON [5] is used for reading the input files.
VTK [6] is used for visualisation during simulations.
Highfive [7] is used to write output files in the HDF5 format.
PETSc [8] is used for its implementation of parrallel (MPI) matrices and vectors (and their associated solvers).

Most of the above libraries are easy to install. However, for PETSc care needs to be taken to enable support for C++, link to the Intel MKL library, and enable support for HDF5. This is enabled through the command:

Install_ICEFEM.sh

```
PETSC_ARCH=optimised
./configure PETSC_ARCH=optimised --with-scalapack-include=$MKLROOT/include --with-
    scalapack-lib="-L$MKLROOT/lib/intel64 -lmkl_scalapack_lp64 -
    lmkl_blacs_intelmpi_lp64" --with-mpi-dir=$I_MPI_ROOT --with-mkl=$MKLROOT --with-
    blas-lapack-dir=$MKLROOT --with-mkl_pardiso-dir=$MKLROOT --with-mkl_cpardiso-dir=$
    MKLROOT --download-hdf5 --download-mumps --with-clanguage=cxx --with-debugging=0 --
    COPTFLAGS='-O3' --FOPTFLAGS='-O3' --CXXOPTFLAGS='-O3'
```

In addition to the environmental variables set by OneMKL and PETSc ($PETSC_DIR, $ PETSC_ARCH, $MKLROOT), an additional environmental variable is used in the cmake files to locate the other libraries, $LIBRARY_DIR, which should point to the parent folder that contains the eigen, rapidjson, and Highfive folders.

1.1.2. Compiling

The code makes uses of cmake to create appropriate makefiles. To compile the code, first run cmake using *cmake .* from the base directory, after which the code itself can be compiled using *make* (or *make -j 8* to compile in parrallel).

1.2. Basic usage

Once compiled, the code can be run using *./IceCode <inputfile>* to run on a single core, or *mpirun -n <number of cores> ./IceCode <inputfile>* to run in parallel. Example input files are included within the folder *TestCases*. Running the code using the example file will print the progress of the simulation to the terminal, and open several plots showing the state of the system. The code will also save the results to a file, which can be visualised using the *PlotData.m* matlab script available in the test cases' folder.

1.3. Input Files

Example input files are provided in the folder *TestCases*, which also includes the required scripts for mesh generation. The input files are in JSON format, containing the following sections:

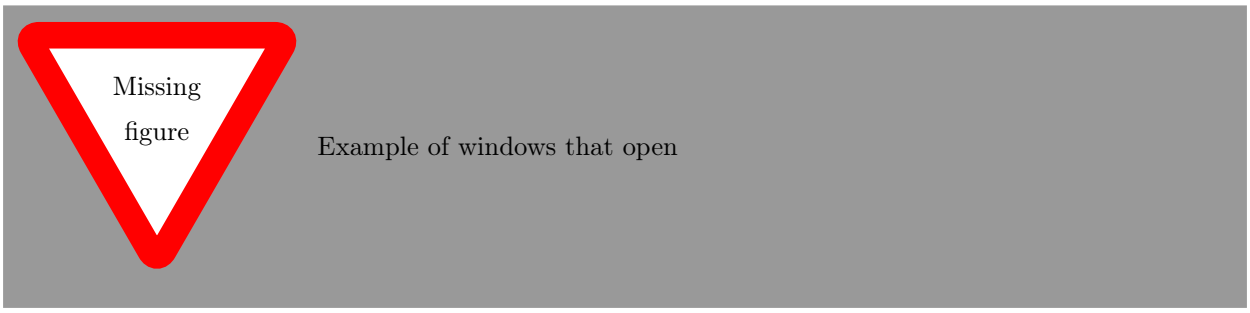

Figure 1:
caption

1.3.1. General settings

Here, the general options, such as whether backup files are generated, and the amount of information printed during simulations are defined:

TestCases/TriAxialCompression/TriAxial.json

```
    "Restart":{
        "Restartable": false,
        "Saveloc": "Results/Backup",
        "SaveFrequency": 100,
        "BackUps": 3
    },
    "Logs":{
        "InfoLevel": 3
    },
```

with *InfoLevel* defining whtehr just simulation progress is printed(1), information within each time step is printed (2), or detailed information about matrix assembly steps are also included (3).

1.3.2. Mesh settings

Mesh generation is done through external scripts, with the path to the mesh file provided in the input file as:

TestCases/TriAxialCompression/TriAxial.json

```
    "mesh":{
        "dim":3,
        "file":"TestCases/TriAxialCompression/Meshgen/Mesh_5mm_p2_c56.h5",
        "ipcount1D": 3
    },
```

This also defines the dimension of the problem (2D or 3D), and the number of integration points used per dimension (e.g. 2 for 2x2 integration points in 2D, and 2 for 2x2x2 integration points in 3D). Note that the mesh generation also pre-calculates the distribution of the elements across the cores, used for parrallelisation, and hence the meshes are specific to the number of cores used.

1.3.3. Material settings

Next, material settings are defined based on the physics they are used within. Starting with the phase-field settings:

TestCases/TriAxialCompression/TriAxial.json

```
    "properties":{
        "PhaseField":{
            "DamFunction": "Quadratic",
```

```
            "DensityDamage": "Step2",
            "GravityDamage": "Step2",
            "ViscoDriving": "None",
            "ViscoDrivingFactor": 0.0,
            "ConsistentEnergy": false,
            "ConsistentDegradation": false,
            "DistributionFunction": "HO_Linear2",
            "k0": 1.0e-12
        },
```

In order of occurance defining the damage functions used for the elastic energy ("Quadratic" or "Cubic"), the degradation functions used for density and gravity (same as for the elastic energy, or "Step2", "Step5", "Step9", or "None"), the degradation function used to define the viscous and plastic driving forces, whether the same phase-field split used for the elastic energy within the phase-field equation is consistently used for the momentum balance as well, the phase-field distribution used ("Linear", "Quadratic", "HO_Linear", or "HO_Quadratic"), and the residual stiffness once the material is fully damaged (i.e. the stiffness of the material once the phase-field variable is zero).

Temperature dependent properties are defined as:

TestCases/TriAxialCompression/TriAxial.json

```
        "Ice":{
            "TemperatureDependent":{
                "TemperatureDependent": true,
                "TRef": 273.15,
                "TOffset": 273.15,
                "QCreep": 150.0e3,
                "ft0": 0.2e6,
                "ftdeg":4.0e4,
                "sydeg": 0.5e6
            },
```

which defines the reference temperature at which the properties are defined, the energy related to viscous creep, the tensile strength at the reference temperature and how much this changes with temperature, and how much the yield strength changes with temperature.

Next, the mechanical properties are defined as:

TestCases/TriAxialCompression/TriAxial.json

```
            "SolidProperties":{
                "Rheology": "DamViscoElasticPlastic",
                "Density": 910,
                "Young": 5.0e9,
                "Poisson": 0.35,
                "G_coss": 1.8e9,
                "l_coss": 2.5e-3,
                "Damping": 1.0e6,
                "A": 1.0e-24,
                "PhiViscLim": 0.2,
                "n": 3.0,
                "sy": 9.5e6,
                "hardening": -0.9,
                "hard_eRef": 2.0e-4,
                "pl_visc": 1.0e3
            },
```

which defines:

- The rheology used for the material, allowing for the following options: "LinearElastic" (only elastic deformations), "ViscoElastic" (elastic and viscous deformations), "ViscoElasticExpl" (elastic and viscous deformations, with an explicit time integration scheme for the viscous creep), "DamViscoElastic" (elastic and viscous deformations, with damage affecting the viscous driving force), "DamViscoElasticExpl" (elastic and viscous deformations, with damage affecting the viscous driving force, and an

explicit time integration scheme), "ViscoElasticPlastic" (elastic, viscous, and plastic deformations), and "DamViscoElasticPlastic" (elastic, viscous, and plastic deformations, with damage affecting the viscous driving force).

- The density, Young's modulus and Poisson's ratio of the solid material.
- The Cosserat shear modulus and length scale.
- Damping coefficient used for the additional damping.
- Glen's law creep coefficient, exponent, and an optional limit beyond which no more creep occurs (for stability).
- The yield strength, and hardening parameters, and viscous regularization used within the Von Mises plasticity model.

The fracture-specific properties:

TestCases/TriAxialCompression/TriAxial.json

```
            "FractureProperties":{
                "Split": "SpecStress",
                "l": 1.5e-3,
                "Gc": 4000.0e-3,
                "Viscosity": 1.0e-1
            },
```

defining the specific phase-field energy split used (using "specStress" for the results used in the paper), phase field legnth scale, fracture energy release rate, and a viscous regularisation parameter. Finally, defining the thermal properties:

TestCases/TriAxialCompression/TriAxial.json

```
            "ThermalProperties":{
                "ThermalConductivity": 2.0,
                "HeatCapacity": 2.1e3
            }
```

1.3.4. Solver settings

For the linear and non-linear solver, we define the degrees of freedom that require solving for, and the staggered solver step they are solved within:

TestCases/TriAxialCompression/TriAxial.json

```
    "Dofs":{
        "DofNames": ["ux","uy","uz", "ox", "oy", "oz","phase","lMult_pf","lMult_pf2","T"],
        "DofStep":  [0,   0,    0,      0,    0,    0,      1,      1,  1,    2]
    },
```

Additionally, we define a block with the solver parameters:

TestCases/TriAxialCompression/TriAxial.json

```
    "nonlinSolver":{
        "max_it": 40,
        "max_outer_it": 1,
        "convCrit": [1.0e-5, 1.0e10, 1.0e-5],
        "tiny": [1.0e-5, 1.0e10, 1.0e-8],
        "linesearch": true,
        "linesearchLims": [0.1, 1.0],
        "LinSolver":{
            "Type": "GMRES",
            "IOptionIdx": [],
            "IOptionVals": []
        },
```

Defining the maximum amount of Newton-Raphson iterations per staggered step, the amount of multi-pass iterations for the staggered solution scheme (1 for a single pass staggered scheme, 2 or more for a multi-pass staggered scheme), and the convergence criteria for the non-linear solver (displacement, force, and energy based thresholds). This also defines whether a line-search algorithm is used, and the limits for the line-search step size. Finally, the type of linear solver is defined, with "GMRES" combining a GMRES iterative solver with a schwarz preconditioner (solving the linear system on each core separately), and "MUMPS" and "PARDISO" using direct LU solver.

TO COMPLETE

2. References

[1] T. Hageman, C. Andrade, E. Martínez-Pañeda, Corrosion of metal reinforcements within concrete and localisation of supporting reactions under natural conditions, Electrochimica Acta 461 (2023) 142624. doi:10.1016/j.electacta.2025.146203.

[2] Microsoft, Windows Subsystem for Linux Documentation, https://learn.microsoft.com/en-us/windows/wsl/.

[3] Accelerate Fast Math with Intel® oneAPI Math Kernel Library, https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl

[4] Libeigen / eigen · GitLab, https://gitlab.com/libeigen/eigen (Apr. 2025).

[5] Tencent/rapidjson, Tencent (Apr. 2025).

[6] W. J. Schroeder, K. Martin, W. E. Lorensen, L. S. Avila, K. W. Martin, B. Lorensen, The Visualization Toolkit: An Object-Oriented Approach to 3D Graphics ; [Visualize Data in 3D - Medical, Engineering or Scientific ; Build Your Own Applications with C++, Tcl, Java or Python ; Includes Source Code for VTK (Supports UNIX, Windows and Mac)], 4th Edition, Kitware, Inc, Clifton Park, NY, 2006.

[7] A. Devresse, N. Cornu, L. Grosheintz-Laval, O. Awile, T. de Geus, F. Pereira, M. Wolf, HighFive Contributors, HighFive - Header-only C++ HDF5 interface, Zenodo (Dec. 2024). doi:10.5281/ZENODO.10679422.

[8] S. Balay, S. Abhyankar, M. Adams, S. Benson, J. Brown, P. Brune, K. Buschelman, E. Constantinescu, L. Dalcin, A. Dener, V. Eijkhout, J. Faibussowitsch, W. Gropp, V. Hapla, T. Isaac, P. Jolivet, D. Karpeev, D. Kaushik, M. Knepley, F. Kong, S. Kruger, D. May, L. McInnes, R. Mills, L. Mitchell, T. Munson, J. Roman, K. Rupp, P. Sanan, J. Sarich, B. Smith, S. Zampini, H. Zhang, J. Zhang, PETSc/TAO Users Manual (Rev. 3.20), Tech. Rep. ANL–21/39 Rev. 3-20, 2205494, 185712 (Nov. 2023). doi:10.2172/2205494.